02 — BCNF, 4NF, and 5NF

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Explain the precise difference between 3NF and BCNF
- Identify multi-valued dependencies and fix them for 4NF
- Understand join dependencies and 5NF
- Know when to stop normalizing (practical limits)
- Recognize when BCNF decomposition is lossless vs. lossy

Boyce-Codd Normal Form (BCNF)

Why BCNF is needed (3NF is sometimes insufficient)

3NF allows a situation where a non-candidate-key attribute determines part of a candidate key. BCNF closes this loophole.

Rule

A table is in BCNF if:

1. It is in 3NF
2. **For every non-trivial functional dependency X → Y, X must be a superkey**

(A superkey is any set of columns that uniquely identifies a row.)

In other words: only keys are allowed to determine anything.

The classic BCNF violation: overlapping candidate keys

```
COURSE_ASSIGNMENTS:
┌────────────┬─────────────────┬──────────────────────┐
│ student_id │ subject         │ teacher              │
```

```
├──────────────┼──────────────────┼──────────────────────┤
│ S01          │ Math             │ Dr. Smith            │
│ S01          │ Physics          │ Dr. Jones            │
│ S02          │ Math             │ Dr. Smith            │
│ S02          │ Physics          │ Dr. Brown            │  ← S02 has different Physics
teacher!
└──────────────┴──────────────────┴──────────────────────┘

Assumptions (business rules):
  - A student takes each subject with exactly one teacher
  - A teacher teaches only one subject

Functional dependencies:
  (student_id, subject) → teacher      ← CK1: composite candidate key
  (student_id, teacher) → subject      ← CK2: another candidate key (teacher teaches
one subject)
  teacher → subject                     ← THIS is the BCNF violation!

teacher is NOT a superkey (a teacher can teach multiple students)
but teacher → subject (a teacher only teaches one subject)

This is in 3NF (no transitive dep via non-key) but violates BCNF!

Problems:
  UPDATE: If Dr. Smith switches from Math to Algebra, must update every row with Dr.
Smith
  INSERT: Cannot record that Dr. Smith teaches Math until a student is enrolled
```

BCNF Fix

```
SPLIT on the violating FD (teacher → subject):

TEACHER_SUBJECTS:              STUDENT_TEACHERS:
┌──────────────────┬─────────┐  ┌──────────────┬──────────────────────┐
│ teacher          │ subject │  │ student_id   │ teacher              │
├──────────────────┼─────────┤  ├──────────────┼──────────────────────┤
│ Dr. Smith        │ Math    │  │ S01          │ Dr. Smith            │
│ Dr. Jones        │ Physics │  │ S01          │ Dr. Jones            │
│ Dr. Brown        │ Physics │  │ S02          │ Dr. Smith            │
└──────────────────┴─────────┘  │ S02          │ Dr. Brown            │
PK: teacher                    └──────────────┴──────────────────────┘
FD: teacher → subject ✓        PK: (student_id, teacher)

Now every FD has a superkey on the left — BCNF achieved!
```

3NF vs BCNF comparison

```
3NF allows:   non-key attribute → prime attribute (part of some candidate key)
BCNF forbids: any non-trivial FD where the left side is not a superkey

3NF ⊃ BCNF (BCNF is stricter)
```

```
Every BCNF table is also 3NF; not every 3NF table is BCNF.

In practice: most well-designed 3NF tables are also BCNF.
BCNF violations only occur with overlapping composite candidate keys.
```

Fourth Normal Form (4NF)

Multi-valued dependencies (MVDs)

A multi-valued dependency X →→ Y means: for a given X, Y takes multiple values **independently** of any other attribute in the table.

4NF Rule

A table is in 4NF if:

1. It is in BCNF
2. **For every non-trivial multi-valued dependency X →→ Y, X is a superkey**

4NF violation example

```
EMPLOYEE_SKILLS_LANGUAGES:
┌─────────────┬─────────────────┬──────────────┐
│ employee_id │ skill           │ language     │
├─────────────┼─────────────────┼──────────────┤
│ E01         │ Python          │ English      │
│ E01         │ Python          │ French       │
│ E01         │ SQL             │ English      │
│ E01         │ SQL             │ French       │  ← Cartesian product!
│ E02         │ Java            │ Spanish      │
└─────────────┴─────────────────┴──────────────┘

PK: (employee_id, skill, language)

Multi-valued dependencies:
  employee_id →→ skill      (employee E01 knows {Python, SQL} independently)
  employee_id →→ language   (employee E01 speaks {English, French} independently)

Skills and languages are INDEPENDENT — knowing E01 speaks French tells you
nothing about which additional skills they have.

Problems:
  Adding a new skill for E01: must add one row per language (2 inserts!)
  Adding a new language for E01: must add one row per skill (2 inserts!)
  Delete: If E01 stops speaking French, must delete multiple rows
```

4NF Fix — decompose into two tables, one per MVD

```
EMPLOYEE_SKILLS:                EMPLOYEE_LANGUAGES:
┌─────────────┬─────────────────┐  ┌─────────────┬──────────────┐
│ employee_id │ skill           │  │ employee_id │ language     │
├─────────────┼─────────────────┤  ├─────────────┼──────────────┤
```

```
| E01         | Python       |   | E01         | English   |
| E01         | SQL          |   | E01         | French    |
| E02         | Java         |   | E02         | Spanish   |
└─────────────┴──────────────┘   └─────────────┴───────────┘

PK: (employee_id, skill)        PK: (employee_id, language)

Now adding E01's new skill "Go" = 1 insert (not 2!)
And adding E01's new language "German" = 1 insert (not 2!)
```

```sql
-- 4NF compliant
CREATE TABLE employee_skills (
    employee_id INTEGER NOT NULL REFERENCES employees(employee_id),
    skill       TEXT    NOT NULL,
    proficiency TEXT    NOT NULL DEFAULT 'intermediate'
                    CHECK (proficiency IN
('beginner','intermediate','advanced','expert')),
    PRIMARY KEY (employee_id, skill)
);

CREATE TABLE employee_languages (
    employee_id INTEGER NOT NULL REFERENCES employees(employee_id),
    language    TEXT    NOT NULL,
    proficiency TEXT    NOT NULL DEFAULT 'conversational'
                    CHECK (proficiency IN
('basic','conversational','fluent','native')),
    PRIMARY KEY (employee_id, language)
);
```

Fifth Normal Form (5NF)

Join dependencies

A **join dependency** exists when a table can only be losslessly decomposed into more than two tables — not into just two tables.

5NF Rule

A table is in 5NF if:

1. It is in 4NF
2. **Every join dependency is implied by the candidate keys**

5NF is also called **Project-Join Normal Form (PJNF)**.

5NF violation example

```
SUPPLIER_PARTS_PROJECTS:
┌──────────┬──────────┬──────────┐
| supplier | part     | project  |
├──────────┼──────────┼──────────┤
| S1       | P1       | J1       |
```

```
| S1       | P1      | J2      |
| S1       | P2      | J1      |
| S2       | P1      | J1      |
└──────────┴─────────┴─────────┘


Business rule:
  A supplier supplies a part, that part is used in a project,
  AND the supplier supplies TO that project — all three must hold simultaneously.
  This is NOT a simple pairwise relationship — it's a ternary relationship.

This table CANNOT be losslessly decomposed into just two 2-way tables.
Attempting to do so and then rejoining creates "spurious" rows.

This is a 5NF violation (join dependency is not implied by the key).

5NF fix: In this case the original ternary table IS the 5NF solution.
  The table is correct as-is if the ternary constraint truly holds.
  5NF tells us: this table cannot be further decomposed without losing info.
```

Normal Forms Hierarchy

```
Normal Form Hierarchy and What Each Eliminates:


  ┌──────────────────────────────────────────────────────────────────┐
  │ 1NF: Non-atomic values, repeating groups                         │
  │  └─► 2NF: Partial functional dependencies                        │
  │       └─► 3NF: Transitive functional dependencies                │
  │            └─► BCNF: Non-superkey functional dependencies        │
  │                 └─► 4NF: Multi-valued dependencies               │
  │                      └─► 5NF: Join dependencies                  │
  │                           └─► 6NF (temporal): Not in SQL         │
  └──────────────────────────────────────────────────────────────────┘


Most practical databases target: 3NF or BCNF
Academic/research systems may target: 4NF or 5NF
6NF is used in temporal databases (rarely in PostgreSQL directly)
```

```
FD type           Violates    Fixed by
──────────────────────────────────────────────────────────────
Partial FD        2NF         Extract the partial dep into table
Transitive FD     3NF         Extract the transit. dep into table
Non-superkey FD   BCNF        Extract the violating FD into table
Multi-valued dep  4NF         Separate tables for each MVD
Join dependency   5NF         (usually already in 5NF if 4NF is met)
```

When BCNF Decomposition Loses Information

The key danger: BCNF decomposition is not always **dependency-preserving**.

```
Original BCNF-violating table:
  (student_id, subject, teacher)
  FDs: teacher → subject
       (student_id, subject) → teacher
       (student_id, teacher) → subject

BCNF decomposition:
  Table A: (teacher, subject)
  Table B: (student_id, teacher)

To check "does student S01 take Math?":
  → Join A and B:
    A: Dr. Smith → Math
    B: S01 → Dr. Smith
    → S01 takes Math (recovered from join)

This is LOSSLESS — the original rows can be reconstructed.

But: the dependency (student_id, subject) → teacher is LOST!
  We can no longer enforce: "a student takes each subject with exactly one teacher"
  without a complicated trigger.

Decision: 3NF allows preserving this constraint; BCNF trades it for less redundancy.
Sometimes 3NF is the right choice over BCNF!
```

SQL Examples

BCNF-compliant course assignment schema

```sql
-- The BCNF-violating situation (for reference only, do not use):
-- CREATE TABLE course_assignments_bad (
--     student_id  INTEGER NOT NULL,
--     subject     TEXT    NOT NULL,
--     teacher     TEXT    NOT NULL,
--     PRIMARY KEY (student_id, subject)
-- );

-- BCNF compliant decomposition:
CREATE TABLE teachers (
    teacher_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        TEXT    NOT NULL,
    subject     TEXT    NOT NULL  -- each teacher teaches exactly one subject
);

CREATE TABLE student_enrollments (
    student_id  INTEGER NOT NULL REFERENCES students(student_id),
    teacher_id  INTEGER NOT NULL REFERENCES teachers(teacher_id),
    enrolled_at DATE    NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (student_id, teacher_id)
);
```

```sql
-- Reconstructing the original view
CREATE VIEW student_subjects AS
SELECT
    se.student_id,
    t.subject,
    t.name AS teacher_name
FROM student_enrollments se
JOIN teachers t ON t.teacher_id = se.teacher_id;
```

4NF-compliant employee skills/languages schema

```sql
-- Already shown above; here is the query to reconstruct the "employee profile"
CREATE VIEW employee_profile AS
SELECT
    e.employee_id,
    e.full_name,
    array_agg(DISTINCT es.skill)     AS skills,
    array_agg(DISTINCT el.language)  AS languages
FROM employees e
LEFT JOIN employee_skills    es ON es.employee_id = e.employee_id
LEFT JOIN employee_languages el ON el.employee_id = e.employee_id
GROUP BY e.employee_id, e.full_name;
```

Common Mistakes

1. **Treating BCNF and 3NF as interchangeable** — They differ when there are overlapping candidate keys. In most tables they coincide, but when they differ, the choice matters.
2. **Decomposing to BCNF when it loses dependencies you need** — Sometimes 3NF is preferable because it preserves the ability to enforce important FDs as simple constraints. BCNF may require triggers instead.
3. **Confusing multi-valued dependencies with functional dependencies** — MVD: X →→ Y (multiple Y values for one X, independent of Z) — FD: X → Y (exactly one Y for each X)
4. **Over-normalizing small tables** — Normalizing a table with 5 rows and 3 columns to BCNF/4NF often creates more problems than it solves.
5. **Confusing 4NF with removing duplicates** — 4NF is about independent multi-valued attributes, not about row duplication.

Best Practices

1. **In practice, aim for 3NF** — it's the right balance for most production systems.
2. **Check for BCNF** when you have overlapping composite candidate keys.
3. **Check for 4NF** when you have attributes that are genuinely independent multi-valued facts of the same entity (skills + languages, colors + sizes, etc.).
4. **Document why** you stopped normalizing — technical debt is manageable when it's conscious.

5. **5NF is rarely needed** in practice — if you think you need it, re-examine whether a ternary relationship is truly necessary.

Performance Considerations

Higher normal forms → more tables → more joins:

```
-- Reconstructing employee full profile (4NF) requires two joins:
SELECT
    e.employee_id,
    e.full_name,
    es.skill,
    el.language
FROM employees e
CROSS JOIN LATERAL (
    SELECT skill FROM employee_skills WHERE employee_id = e.employee_id
) es
CROSS JOIN LATERAL (
    SELECT language FROM employee_languages WHERE employee_id = e.employee_id
) el;
-- This generates the Cartesian product at query time — which is fine because
-- that's the semantically correct result for this data

-- With proper indexes, this is still fast:
CREATE INDEX idx_emp_skills_emp   ON employee_skills (employee_id);
CREATE INDEX idx_emp_langs_emp    ON employee_languages (employee_id);
```

Interview Questions & Answers

Q1: What is the difference between 3NF and BCNF?

A: 3NF allows a functional dependency X → Y where Y is part of a candidate key (a "prime attribute") and X is not a superkey. BCNF forbids this — every non-trivial FD must have a superkey on the left side. BCNF is stricter. In practice, violations only occur with overlapping composite candidate keys.

Q2: Can a table be in 3NF but not BCNF? Give an example.

A: Yes. The classic example is the course assignment table: (student_id, subject, teacher) where two FDs hold: (student_id, subject) → teacher, and teacher → subject. The second FD violates BCNF (teacher is not a superkey) but does not violate 3NF (subject is a prime attribute — part of a candidate key).

Q3: What is a multi-valued dependency?

A: X →→ Y means for each value of X, there is a set of Y values that is independent of all other attributes in the table. Example: employee_id →→ skill and employee_id →→ language — an employee's skills and languages are independent sets, requiring a Cartesian product to fully represent.

Q4: Why is 4NF a problem even though the data appears correct?

A: The table appears to store correct data, but it stores every combination of independent facts (Cartesian product), creating update anomalies: adding one skill requires adding N rows (one per language); deleting one language requires deleting M rows (one per skill). The storage is also redundant.

Q5: What is a lossless decomposition?

A: A decomposition is lossless if you can always reconstruct the original table exactly by joining the decomposed tables, with no spurious (phantom) rows added. BCNF decompositions are always lossless but may not preserve all functional dependencies.

Q6: When would you choose 3NF over BCNF?

A: When BCNF decomposition would lose a functional dependency that you need to enforce, and enforcing it via triggers would be too complex or fragile. 3NF allows you to preserve the dependency as a natural constraint of the table structure.

Q7: How common is a real 4NF violation in practice?

A: Moderately common in HR and profile systems (employees with multiple skills AND multiple languages), product catalogs (products with multiple colors AND sizes), and configuration tables (modules with multiple versions AND environments). Recognizing the Cartesian-product smell in junction tables is the key skill.

Q8: Is there a "6NF" and what is it for?

A: Yes, 6NF (sometimes called Domain-Key Normal Form applied temporally) is used in temporal databases. A table in 6NF has no non-trivial join dependencies — every table contains exactly one fact (the key and one non-key attribute). In practice, it's used in bitemporal data modeling to handle historical/validity-period data cleanly.

Exercises with Solutions

Exercise 1

The following table has employees with multiple certifications and multiple project assignments. Identify the normal form violation and fix it.

```
EMPLOYEE_CERTS_PROJECTS(employee_id, cert_name, project_name)
```

Solution:

This violates 4NF. The multi-valued dependencies are:

```
employee_id →→ cert_name     (an employee has multiple certs, independent of projects)
employee_id →→ project_name  (an employee has multiple projects, independent of certs)
```

Fix:

```
CREATE TABLE employee_certifications (
    employee_id INTEGER NOT NULL REFERENCES employees(employee_id),
    cert_name   TEXT    NOT NULL,
    issued_date DATE,
    PRIMARY KEY (employee_id, cert_name)
);

CREATE TABLE employee_projects (
    employee_id INTEGER NOT NULL REFERENCES employees(employee_id),
    project_id  INTEGER NOT NULL REFERENCES projects(project_id),
    role        TEXT    NOT NULL DEFAULT 'member',
```

```
    PRIMARY KEY (employee_id, project_id)
);
```

Exercise 2

Design the BCNF decomposition for: `TEACHES(professor, course, room)` with FDs:

- `(professor, course) → room`
- `room → course`

Solution:

`room → course` violates BCNF (room is not a superkey).

Decomposition:

```
CREATE TABLE room_courses (
    room    TEXT NOT NULL PRIMARY KEY,
    course  TEXT NOT NULL  -- each room is dedicated to one course
);

CREATE TABLE professor_rooms (
    professor TEXT NOT NULL,
    room      TEXT NOT NULL REFERENCES room_courses(room),
    PRIMARY KEY (professor, room)
);
-- Reconstruct: JOIN professor_rooms WITH room_courses
```

Cross-References

- `01_normalization_1nf_2nf_3nf.md` — prerequisite material
- `03_denormalization.md` — controlled violation of normal forms
- `04_entity_relationships.md` — ternary relationships (5NF context)
- `../05_PostgreSQL_Core/08_constraints.md` — enforcing dependency rules